

→ Tags:-

html
head
body

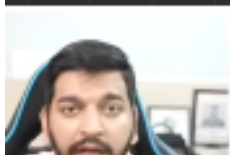
→ what?

→ markup entity

→ structure

→ meaning

→ formatting



→ Semantics

→ `<h1>` → `<h6>`

→ `<p>`

→ `<a>`

→ `<ul>` `<ol>`

→ `<table>` `<tr>` `<td>`

`<img>`

`<form>`

`<input>`

Structure

→ `html`

→ `head` `body`

→ `<header>`

→ `<main>`

→ `<aside>`

→ `<footer>`

→ `<article>`

→ `<section>`

→ `<nav>`

formatting

→ `<b>`

→ `<i>`

→ `<u>`

→ `<u>`

→ `<strong>`

→ `<em>`

→ `<sup>` `<sub>`

→ `<pre>`

→ `<code>`

EXPLORER

...

Welcome

index.html X

LECTURE5WD

index.html

index.html > html > body > a

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta http-equiv="X-UA-Compatible" content="IE=edge">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Document</title>

</head>

<body>

<h1>This is my First Heading</h1>

<h2>This is my second heading</h2>

<h3>This is my third heading</h3>

<h4>This is my fourth heading</h4>

<h5>This is my fifth heading</h5>

<h6>This is my sixth heading</h6>

<p>This is my first para</p>

Click here

</body>

</html>



OUTLINE

23
24
25
26
27
28
29
30
31
32
33
34
35

```
<ul>
  <li>Maggi</li>
  <li>Kurkure</li>
  <li>Lays</li>
</ul>

<ol>
  <li>Paani garam karo</li>
  <li>Chap patti daaloi</li>
  <li>CHaai peelo</li>
</ol>
```

```
<table>
  <tr>
    <td>Sr. no</td>
    <td>Name</td>
    <td>Login</td>
  </tr>
  <tr>
    <td>1</td>
    <td>2</td>
    <td>3</td>
  </tr>
  <tr>
    <td>4</td>
    <td>5</td>
    <td>6</td>
  </tr>
</table>
```

EXPLORER

...

Welcome

index.html X

LECTURE5WD

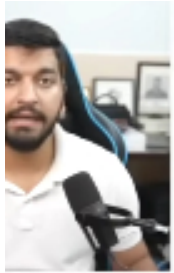
index.html

index.html > html > body > table > tr

```
37     <tr>
38         <td>Sr. no</td>
39         <td>Name</td>
40         <td>Login</td>
41     </tr>
42     <tr>
43         <td>1</td>
44         <td>2</td>
45         <td>3</td>
46     </tr>
47     <tr>
48         <td>4</td>
49         <td>5</td>
50         <td>6</td>
51     </tr>
52 </table>
53
54 table>(tr>td*3)
55
56
57
58 </body>
59 </html>
```

OUTLINE





html > body
 </table>

```

```

```
<p>Lorem <i>ipsum</i> dolor sit, amet consectetur adipisicing elit. Labore ipsa assumenda tempore, a
```

```
<p><s>Lorem</s> ipsum dolor sit amet consectetur adipisicing elit. Adipisci temporibus, sunt delectu
```

```
<p>  
  2<sup>5</sup>  
</p>
```

The sub element represents a subscript.
[MDN Reference](#)

```
<p> 0<sub>2</sub> </p>
```

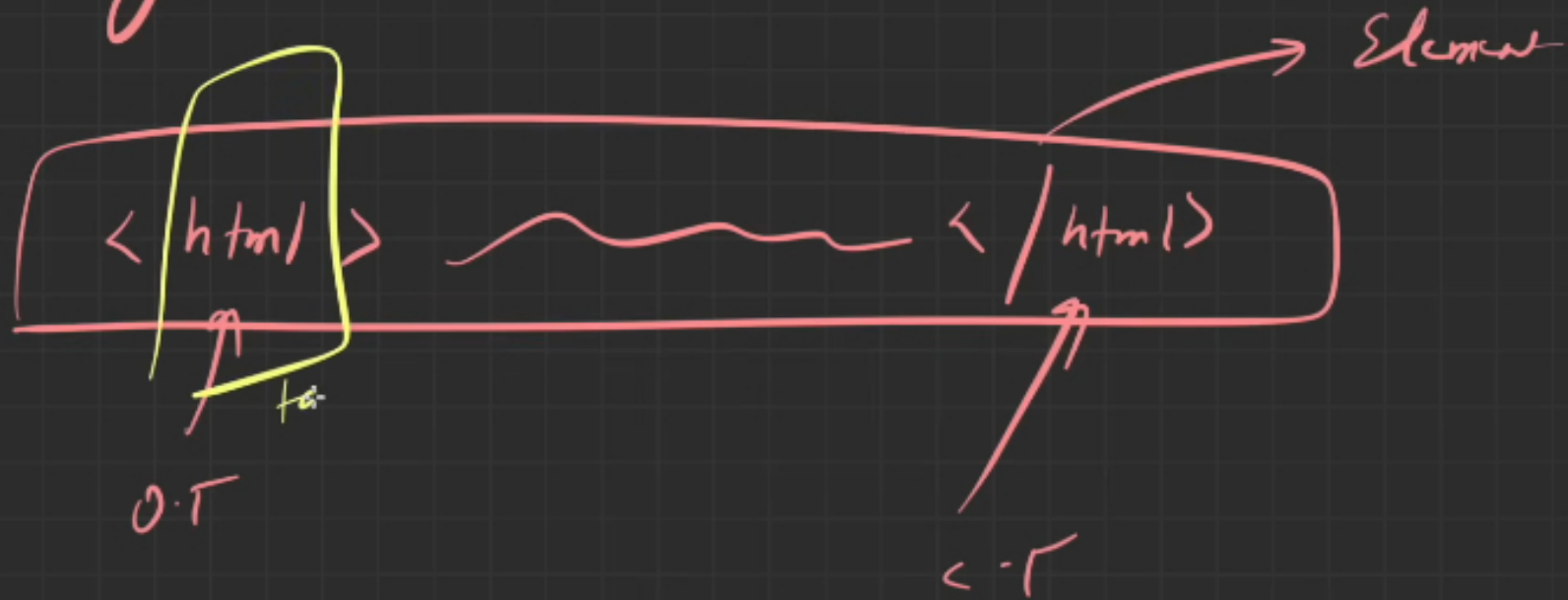
```
<pre>  
  My  
  Name  
  is  
  Love  
  Babbar,  
  I live in Delhi  
</pre>
```

```
</body>
```

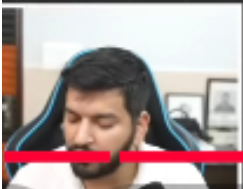
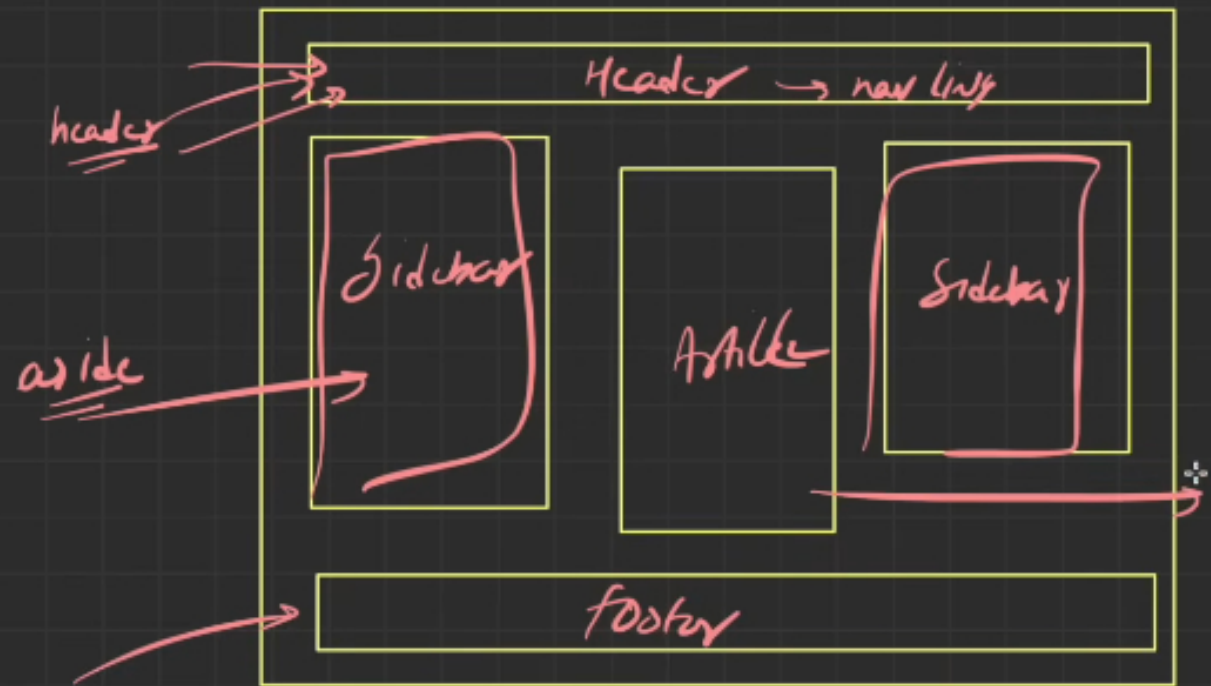
```
</html>
```

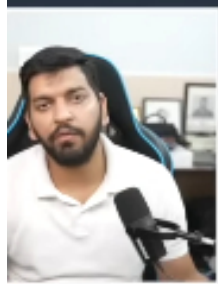


Tags vs Element



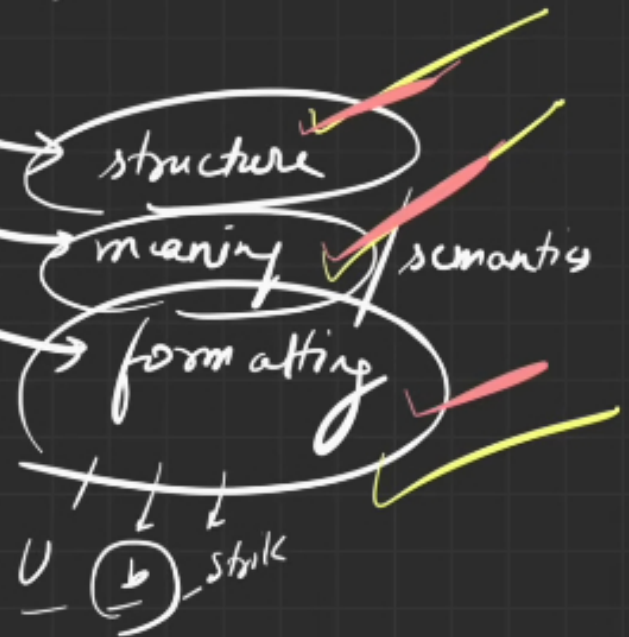
→ Structured tag





html
head
body

markup entity



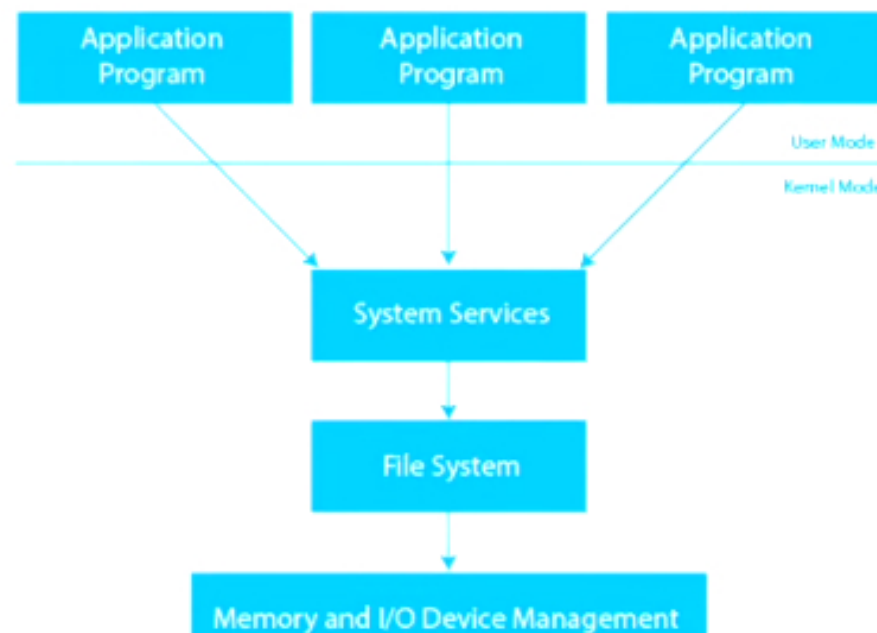
- Most asked → OS core conc

SECTION 1 - INTRODUCTION TO OPERATING SYSTEMS

Definition

An **Operating System (OS)** is system software that acts as an **intermediary between hardware and the user/application software**. It manages hardware resources and provides a platform for programs to run.

"An OS is a resource allocator and a control program."



Custom
20 -
X X
Addi
adv
Mac
Wind
Linux



Why Do We Need an OS?

- Without OS, every program would need to directly manage CPU, RAM, Disk, I/O
- OS abstracts hardware complexity
- Provides security, multi-tasking, memory management, file handling

→ directly → App

Goals of an OS

Primary Goal --> Convenience (easy to use)

Secondary Goal --> Efficiency (use hardware well)

Functions of OS

- +-----+
- | | |
|------------------------|---|
| Process Management | --> Create, schedule, terminate processes |
| Memory Management | --> Allocate / deallocate RAM |
| File System Management | --> Files, directories, permissions |
| I/O Management | --> Device drivers, buffering |
| Security & Protection | --> Auth, access control |
| Network Management | --> Protocols, sockets |

+-----+





YouTube

cs core for placement



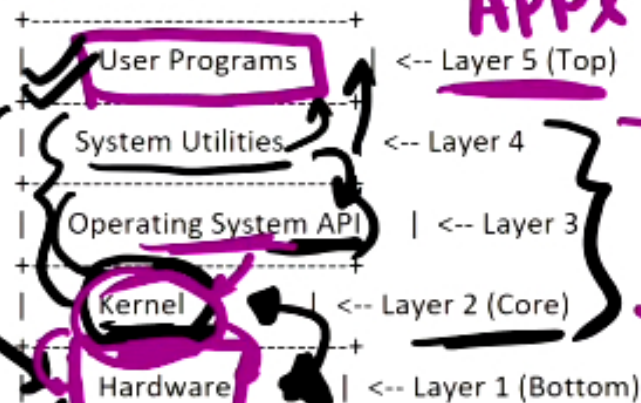
+ Create



*17-18 sec:

SECTION 2 - ARCHITECTURE OF AN OPERATING SYSTEM

Layered Architecture



Kernel

- The core of the OS
- Always resident in memory
- Handles: CPU scheduling, memory, I/O, inter-process communication (IPC)

Imp. ↓↓

on (X) Reboot
up (O)

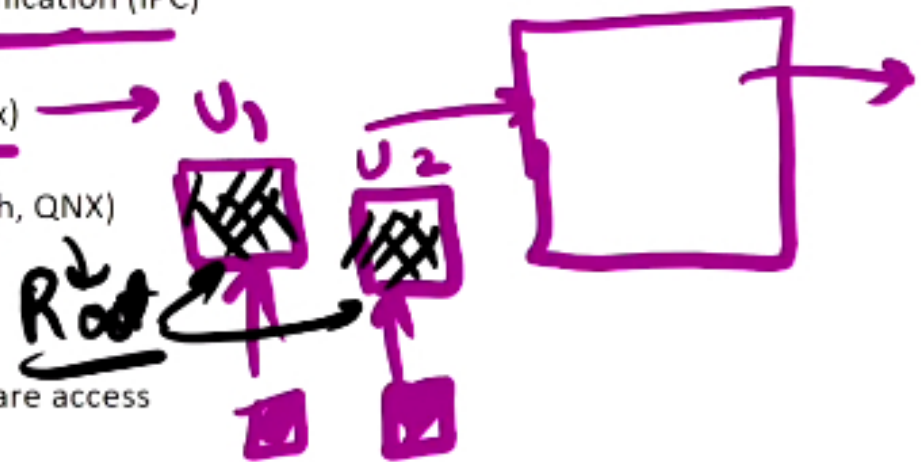
core



• Handles: CPU scheduling, memory, I/O, inter-process communication (IPC)

Types of Kernel

- Monolithic Kernel --> All OS services in one large block (Linux, Unix)
Fast but less modular
- Microkernel --> Minimal kernel; services run in user space (Mach, QNX)
Slower (message passing) but more stable/secure
- Hybrid Kernel --> Mix of both (Windows NT, macOS XNU)
Balance of performance and modularity
- Exokernel --> Extremely minimal; gives programs direct hardware access
Used in research OSes



System Calls

- **Definition:** Interface between a user program and the OS. When an app needs a privileged operation, it makes a **system call**.
- Examples: fork(), exec(), read(), write(), open(), close(), exit()

App --> System Call --> Kernel Mode --> Hardware
(trap/interrupt)

Used in research Uses

System Calls

NASA:

- **Definition:** Interface between a user program and the OS. When an app needs a privileged operation, it makes a **system call**.
- Examples: `fork()`, `exec()`, `read()`, `write()`, `open()`, `close()`, `exit()`

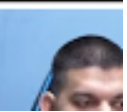
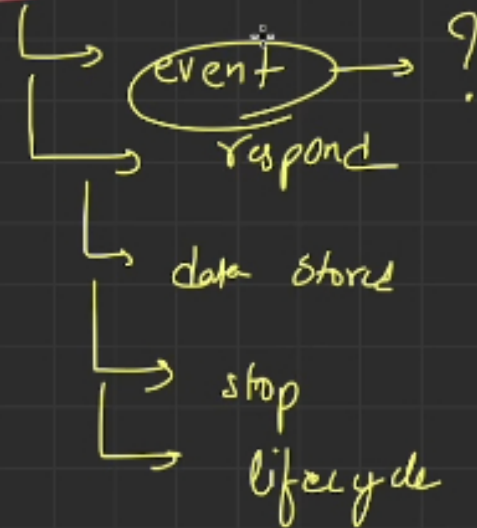
App --> System Call --> Kernel Mode --> Hardware
(trap/interrupt)

Types of System Calls

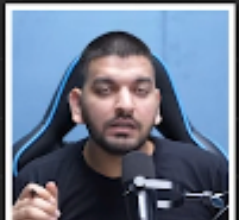
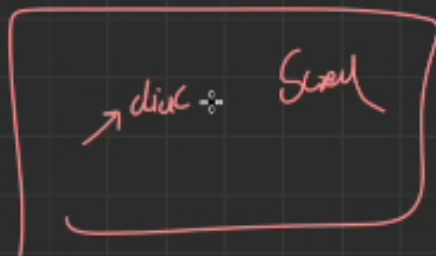
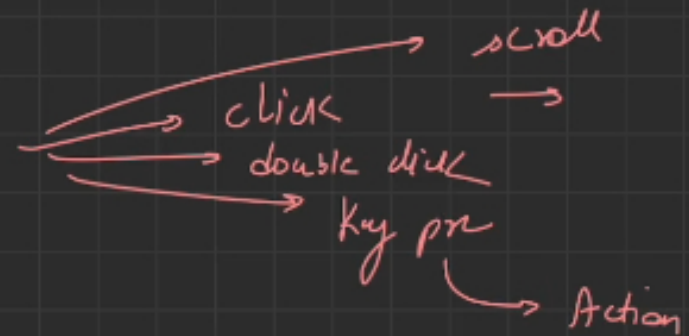
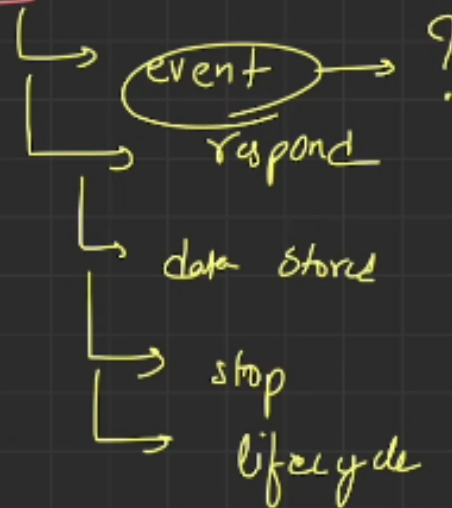
1. Process Control --> `fork`, `exit`, `wait`, `exec`
2. File Management --> `open`, `read`, `write`, `close`
3. Device Management --> `ioctl`, `read`, `write`
4. Information --> `getpid`, `alarm`, `sleep`
5. Communication --> `pipe`, `socket`, `shmget`



→ Browser Events :-



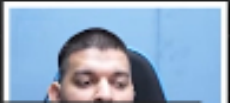
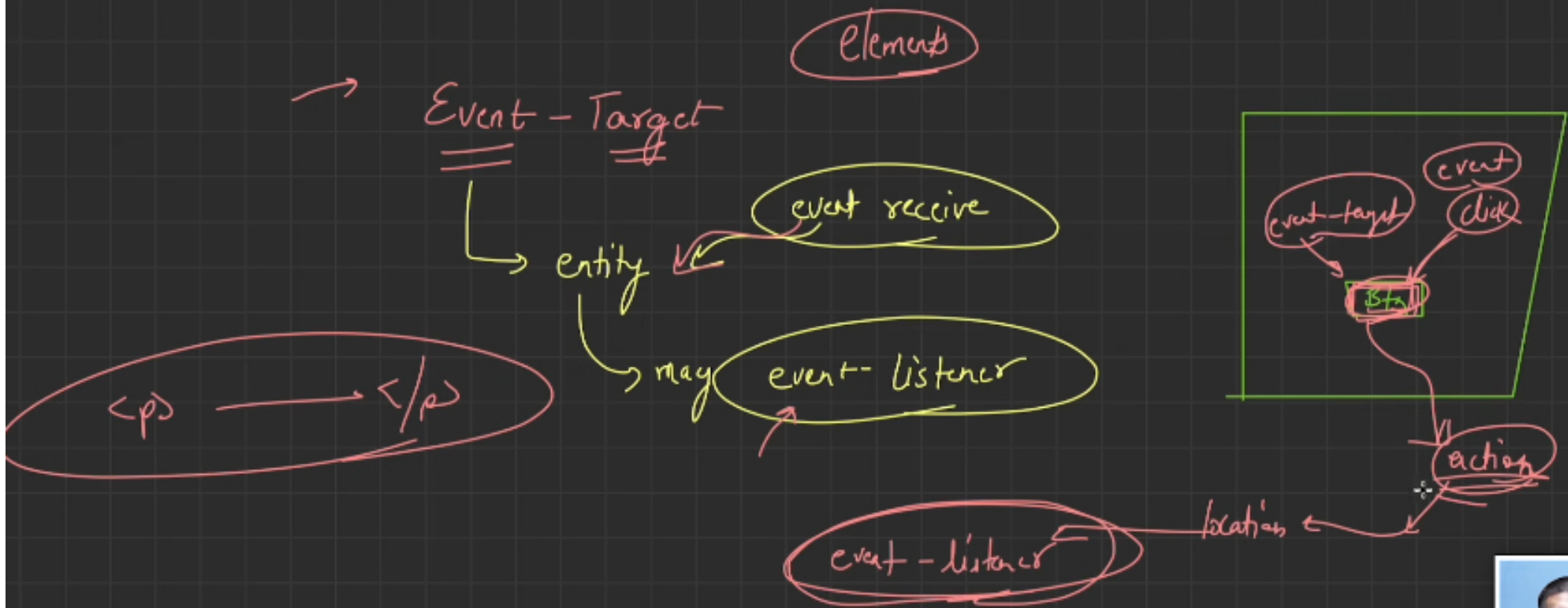
→ Browser Events :-



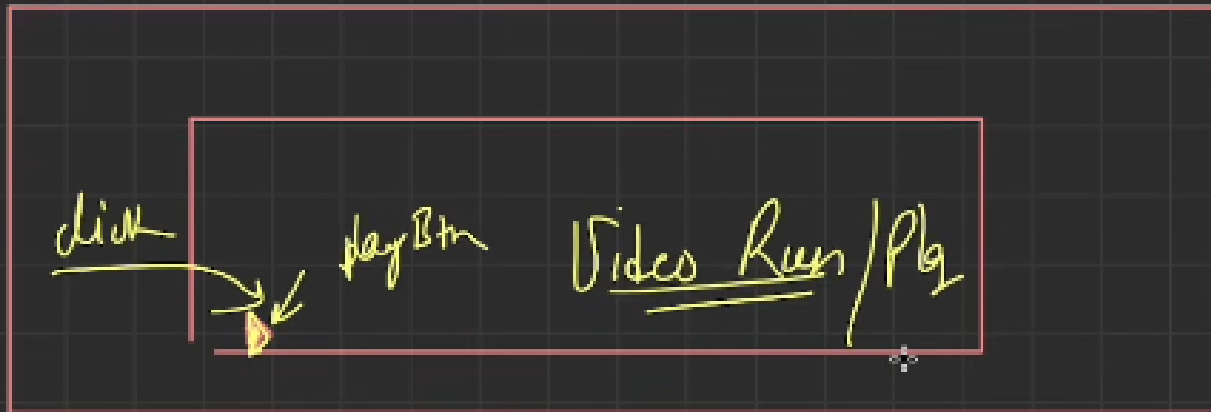
```
> monitorEvents(document);
```

```
< undefined
```

```
> |
```



event - listener



event → 'click'

play Btn → event target

Video Run/Play → Action/
behavior

event - listener

event-listener ← behaviour

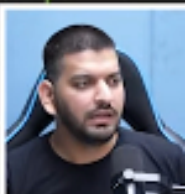
elements

add Event Listener ()

remove Event Listener ()

Syntax:-

<event-target> . add Event Listener (<event=type> , <function → action>)



localhost / js index.js / ...

```
1 let fpara = document.getElementById('fpara');  
2 fpara.textContent = "Hello Babbar"
```

X

Hello Babbar

```
function changeText() {  
  let fpara = document.getElementById('fpara');  
  fpara.textContent = "Hello Babbar"  
}
```

```
let fpara = document.getElementById('fpara');  
fpara.addEventListener('click', changeText);
```

Hello Jee

ecture53 > `js index.js` > ...

```
1
2
3 function changeText() {
4     let fpara = document.getElementById('fpara');
5     fpara.textContent = " Babbar"
6 }
7
8 let fpara = document.getElementById('fpara');
9
10 fpara.addEventListener('click', changeText);
11
12 //fpara.removeEventListener('click', changeText);
```

Hello Jee

Phase of Event

Capturing
At-Target
Bubbling

<div>

< article)

 $\langle h_1 \rangle$ $\rightarrow \langle 1/h \rangle$ $\langle p \rangle$

→ $\langle \rho \rangle$

</article>

 $\langle \delta v \rangle$

click

Capturing phase

A+-Target

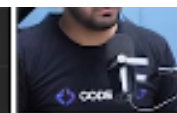
Capturing Phase

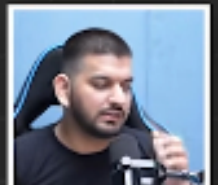
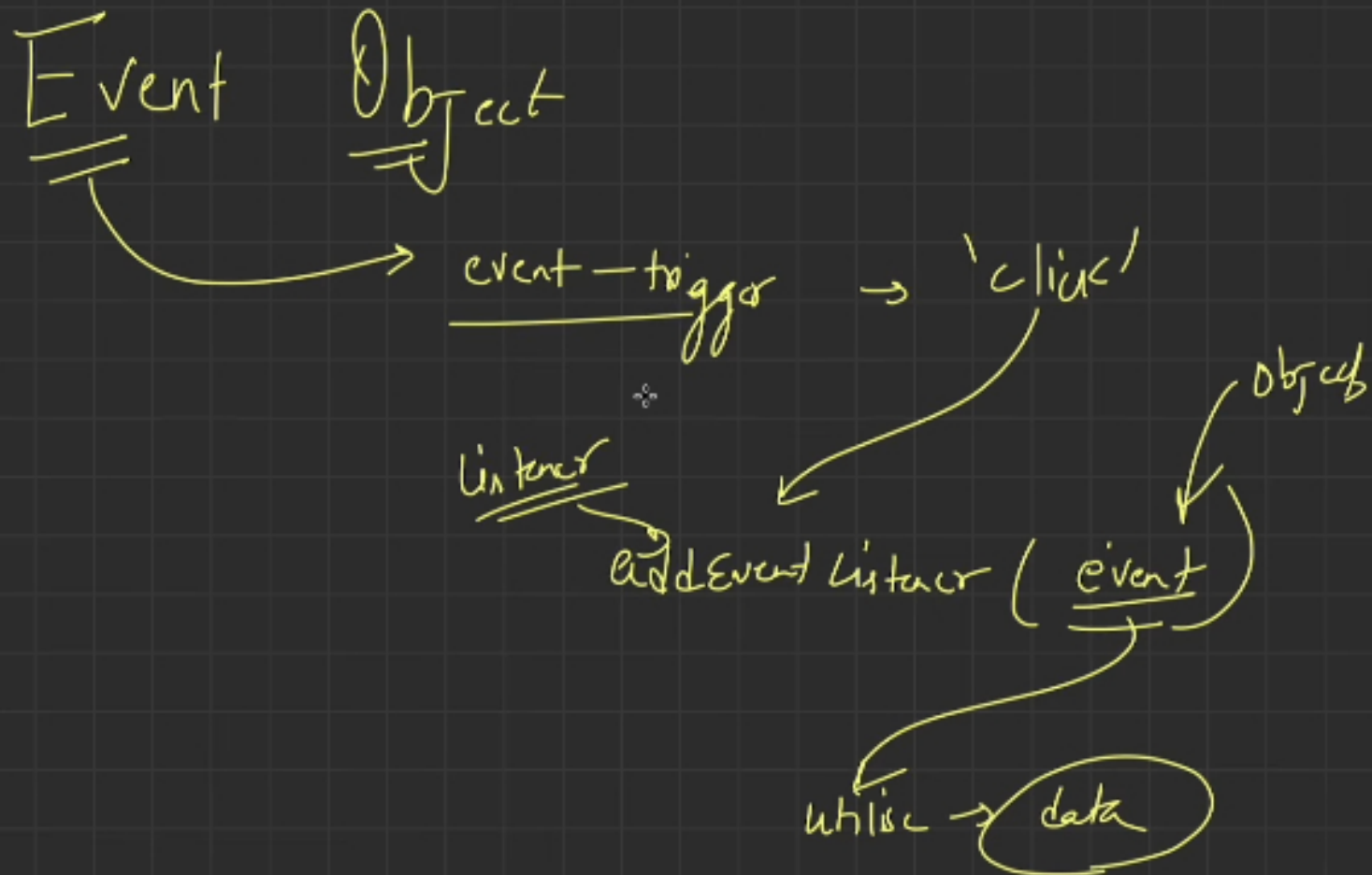
add Event Listener ()

by default

true → useCapture

Bubbling phase





1853 / 15 index.js / 15 ChangerText

```
1
2
3 function changeText(event) {
4     console.log(event);
5     let fpara = document.getElementById('fpara');
6     fpara.textContent = " Babbar"
7 }
8
9 let fpara = document.getElementById('fpara');
10
11 fpara.addEventListener('click', changeText);
12
13 //fpara.removeEventListener('click', changeText);
```

Babbar

```
let anchorElement = document.getElementById('fanchor');  
  
anchorElement.addEventListener('click', function(event) {  
    event.preventDefault();  
    anchorElement.textContent = "Click done bhai"  
});
```

```
</head>  
<body>
```

```
  <div id="wrapper">
```

```
    <p id="fpara">First para</p>
```

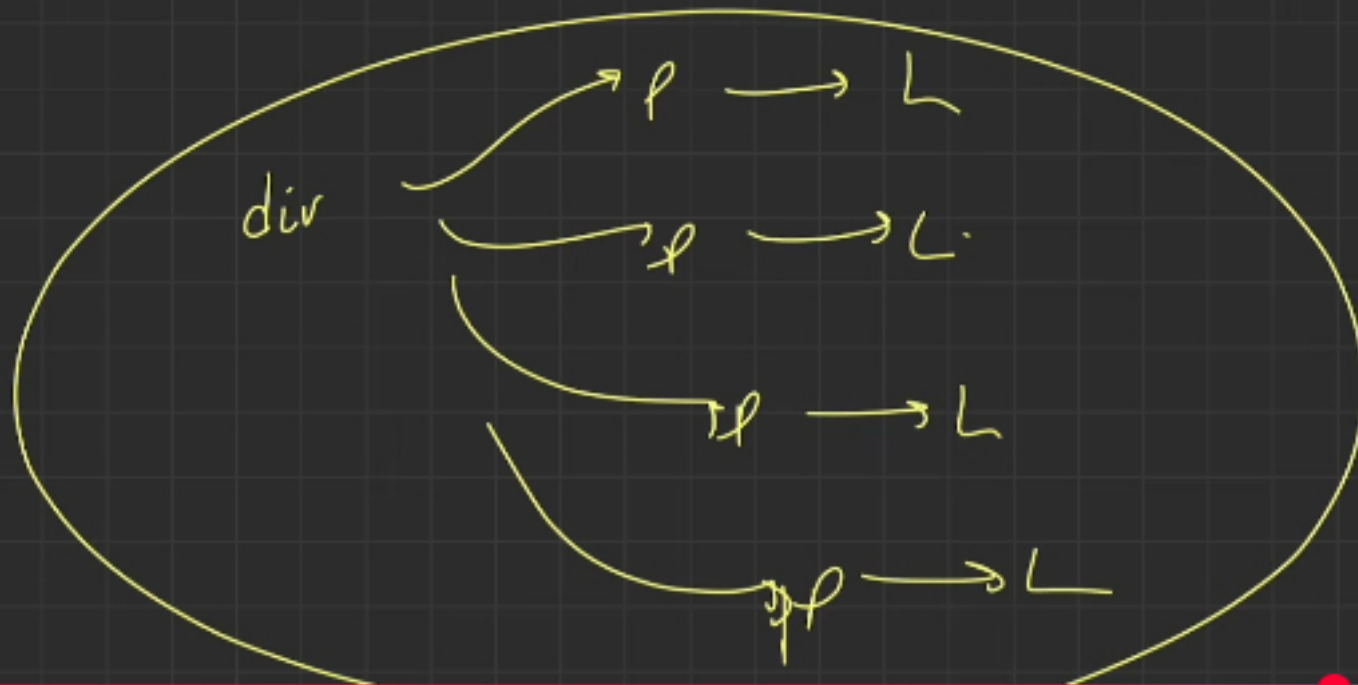
```
    <p id="spara">Second Para</p>
```

```
    <p id="tpara">Third Para </p>
```

```
    <p id="fopara">Fourth Para</p>
```

```
  </div>
```

→ Avoiding too many listeners:-



```
21
22 let paras = document.querySelectorAll('p');
23
24 for(let i=0; i<paras.length; i++) {
25     let para = paras[i];           (local function)(this: HTMLParagraphElement): void
26     para.addEventListener('click', function()-{
27         alert("You have Clicked on para: " + (i+1));
28     })
29 }
```

Fourth